NOTES · PART 4

TEAMS, CODING, VOICE AND LOCAL AGENTS

Memorandum on Agentic AI Applications · 24 Oct 2026 · continues Part 3

Contents

XIV. Inside a team of agents

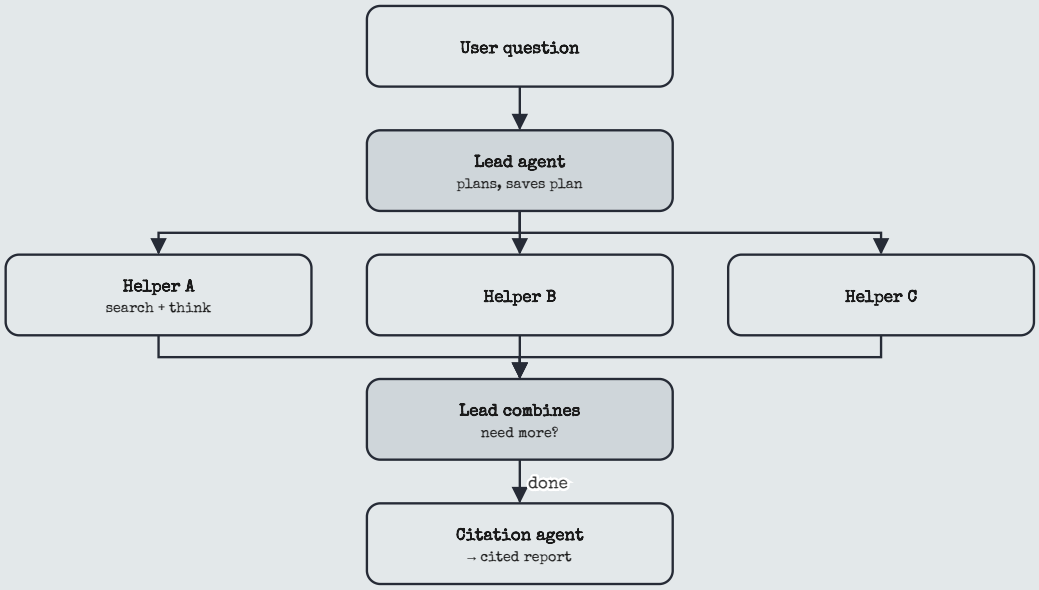
XV. Inside coding agents

XVI. Inside voice and local agents

Diagram key: a shaded box is the AI model or agent, a red dashed box is a risk or a guard, plain boxes are people, tools, data and results. A dashed arrow means "go back and repeat".

XIV. INSIDE A TEAM OF AGENTS

PATTERN	WHO IS IN CHARGE	USE IT FOR
Handoff	Control moves to the specialist, who talks to the user	Support desk: billing, tech, refunds
Manager (agents as tools)	The main agent keeps control, borrows answers	One voice to the user, many helpers
Lead + parallel helpers	A lead plans, helpers search at the same time	Research, comparisons
Conversation (AutoGen)	Agents message each other until done	Write code, run it, fix it
Crew with procedures (MetaGPT)	Each role writes a document for the next	Small software projects



Lead and parallel helpers, as in Anthropic's Research feature.

- A handoff is just a tool named like `transfer_to_billing`. Calling it swaps which agent is in charge.
- Anthropic's team: lead plus helpers beat a single top model by 90% on their research test, but used about 15 times the tokens of a normal chat. Worth it only for valuable, splittable work.
- How much effort to spend: a simple fact needs 1 agent and 3 to 10 tool calls. A comparison needs 2 to 4 helpers. A big topic needs 10 or more.
- Crews work better when roles pass documents (plan, design, task list) instead of chatting. MetaGPT reached 85.9% on a standard coding test this way.
- The 2023 AutoGen code is out of date. Microsoft rebuilt it and now points to Microsoft Agent Framework.

READ THE PAPER

- Handoffs (OpenAI Agents SDK docs): transfer tools
- How we built our multi-agent research system (Anthropic): the real costs and lessons
- AutoGen (Wu et al., 2023): agents that talk
- MetaGPT (Hong et al., 2023): roles and documents

XV. INSIDE CODING AGENTS

1. How coding agents are tested (SWE-bench)

1. Take real fixed bugs from 12 popular Python projects: 2,294 tasks.
2. Give the agent the bug report and the code from before the fix.
3. The agent writes a patch.
4. Run the project's tests: the ones that failed must now pass, and the ones that passed must still pass.
5. At launch the best model fixed under 2%. A cleaned-up set of 500 human-checked tasks ("Verified") is now the usual scoreboard.

2. The commands a good coding agent gets (SWE-agent)

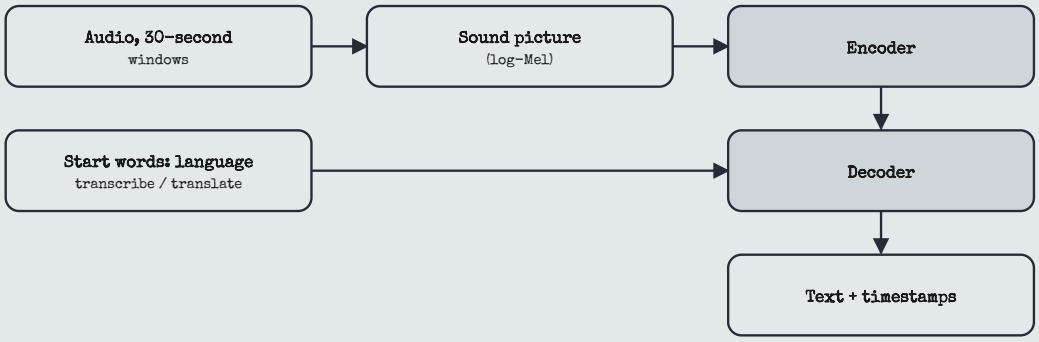
COMMAND	WHY IT HELPS
find_file, search_dir	Short summary of matches (max 50), not a flood
open, goto, scroll	Shows 100 numbered lines at a time
edit start:end	Replaces lines; a syntax checker rejects broken edits
(history)	Old outputs are folded to one line to save space
submit	Turns the changes into a patch

- With these commands, GPT-4 Turbo fixed 12.5% of the full test, against 3.8% before. Today's agents score far higher, but the lesson stays: design the agent's commands like a good UI.

> READ THE PAPER
>
> ▸ SWE-bench (Jimenez et al., 2023): the test
>
> ▸ SWE-agent (Yang et al., 2024): the commands

XVI. INSIDE VOICE AND LOCAL AGENTS

1. Inside the speech-to-text step (Whisper)



Whisper: one model for many speech jobs, picked by special start words.

- Trained on 680,000 hours of real internet audio, so it copes with accents and noise. It made about 55% fewer errors than a model trained on clean audio.
- Weak spots: it can invent words during silence and repeat itself on long audio. Cut out silence first.

2. The two ways to build a voice agent

	CHAINED	SPEECH-TO-SPEECH
Flow	speech-to-text → text agent → text-to-speech	one realtime model, audio in and out
Good at	control, transcripts, reuse your text agent	speed, natural voice, hears tone
Watch out	delay adds up at each stage, so stream each one	less control over exact words, costs more
Start here if	you need Hindi support, logs, strict rules	speed matters most

3. How a local model fits on your laptop

SIZE OF MODEL	4-BIT (Q4) MEMORY, ROUGHLY	8-BIT (Q8) MEMORY, ROUGHLY
3B	about 2 GB	about 3.5 GB
7 to 8B	about 5 GB	about 8.5 GB
13 to 14B	about 8.5 GB	about 15 GB

- Rule of thumb: about 0.6 GB per billion parameters at 4-bit, plus extra for long chats.
- llama.cpp is the engine (GGUF files, CPU or GPU). Ollama wraps it with easy commands and a local API at `localhost:11434` that works with OpenAI-style code.
- Good split: a local model for cheap steps (sorting, pulling out fields), a cloud model for hard steps.

READ THE PAPER

- Whisper (Radford et al., 2022): inside speech-to-text
- Voice agents guide (OpenAI docs): chained vs speech-to-speech
- Ollama (GitHub): easy local models
- llama.cpp (GitHub): the engine

* * *

Continued in Part 5.